

Predicting Initial Coin Offering Success Using Machine Learning: A Comparative Analysis of Classification Models

May 2026

1 Executive Summary

This report develops machine learning models to predict whether an Initial Coin Offering (ICO) fundraising campaign will successfully reach its funding goal, using a dataset of 5,828 campaigns sourced from the IcoBench platform.

The data was cleaned to remove errors, outliers and missing values, and feature engineering and selection was implemented to produce a dataset ready for modelling. This included merging the provided dataset with Bitcoin price at the start date of the ICO campaign, as a proxy for cryptocurrency market sentiment. After data preparation, the dataset consisted of 4475 rows, the target variable *goal*, and 19 predictor variables.

Four machine learning algorithms were trained and evaluated: K-Nearest Neighbour, Decision Tree, Random Forest and Support Vector Machine. All models were tuned using 10-fold cross-validation with AUC as the selection criterion, given the class imbalance in the dataset (39% successful campaigns). Final performance was evaluated on a held-out test set.

All four models demonstrated genuine predictive ability, achieving AUC scores between 0.719 and 0.732. The random Forest model achieved the highest AUC, making it recommended for ranking ICO campaigns by probability of success, for example to enable investors to create a shortlist of projects. The SVM model achieved the highest classification metrics at the standard 0.5 decision threshold, indicating this model may be preferable when a simple success or failure prediction for an ICO is desired.

The strongest predictors of ICO outcome were identified as expert rating of the project, the price of Bitcoin on the launch date, campaign duration, and team size, with successful ICOs generally exhibiting larger teams, shorter durations, lower Bitcoin price at launch and higher expert ratings. Binary variables indicating the presence of website, LinkedIn, GitHub and whitepaper URLs also exhibited a positive relationship with project success, indicating transparency is beneficial to ICO performance.

Limitations of this report include bias towards predicting failure consistent across models due to class imbalance, data quality constraints resulting in substantial missingness, and the exclusion of additional variables such as perceived management confidence, as utilised by Huang et al. (2021), or overall economic data, which future work could incorporate to improve predictive power.

2 Business Understanding

Initial Coin Offerings (ICOs) are blockchain fundraising mechanisms in which companies raise capital by selling digital tokens to investors, which can then be used within the company's platform, or traded on online marketplaces. ICOs operate on an all-or-nothing basis: the company sets a fundraising goal and timeframe, if it is not met the project is considered a failure and they receive nothing. This structure creates a high-stakes environment for both entrepreneurs and investors, underscoring the importance of understanding why ICOs succeed or fail, and the ability to predict a project's outcome.

3 Data Understanding

The analysis in this report is produced using the provided dataset, sourced from IcoBench, containing information about 5828 ICOs and 22 variable columns, summarised in Table ?? below.

In this analysis, *goal* serves as the target variable, and the rest act as potential predictors.

As this is a real world data set, there are a number of inconsistencies and missing data. The vast majority (99.75%) of rows contain missing data, the distribution of which is visualised in Figure 1.

Variable	Description	Data Type
country	Country in which the company headquarters are located.	Text
goal	An indicator variable set to '1' if the project achieved their goal; '0' otherwise.	Numerical
ico_start	The start date of the token offering (yyyy-mm-dd).	Text
ico_end	The end date of the token offering (yyyy-mm-dd).	Text
price_usd	Price per token (USD).	Numerical
distributed_in_ico	The percentage of blockchain coin offered to investors, rather than to the fundraising team (%).	Numerical
sold_tokens	Number of tokens sold during offering period.	Numerical
token_for_sale	Number of tokens released for offering period.	Numerical
whitelist	An indicator variable set to '1' if the project used a whitelist process (approved list of investors who get priority); '0' otherwise.	Text
kyc	An indicator variable set to '1' if the project used a Know-Your-Customer process (investor verification process); '0' otherwise.	Text
bonus	An indicator variable set to '1' if the project has information on bonuses provided to token buyers; '0' otherwise.	Text
mvp	An indicator variable set to '1' if the Minimum Viable Product of the project was available at the start of the offering; '0' otherwise.	Text
pre_ico_start	The start date of the presale round (yyyy-mm-dd).	Text
pre_ico_end	The end date of the presale round (yyyy-mm-dd).	Text
pre_ico_price_usd	Token price at the presale round (USD).	Numerical
link_white_paper	URL to project's white paper.	Text
linkedin_link	URL to project's LinkedIn page.	Text
github_link	URL to project's GitHub page.	Text
website	URL to project's website.	Text
rating	Quality rating score given by investment experts prior to ICO launch; ranges from 1 (very poor) to 5 (very good).	Numerical
teamsize	Total number of team members associated with the project.	Numerical
ERC20	An indicator variable set to '1' if tokens were created under the ERC-20 standard; '0' otherwise.	Text

Table 1: Description and data type of each variable in the raw ICO dataset.

The variable `sold_tokens` consists of 97% missing values, and `pre_ico_price_usd` similarly displays a high level of missingness with around 74% values being NA. This indicates that these variables will not be useful features in building a predictive model. Other variables displaying missing values are `price_usd`, `distributed_in_ico`, `token_for_sale`, `rating` and `teamsize`, however the proportion of missingness in these predictors is much less severe, varying from around 11-28%.

The class distribution of the binary variables, `goal`, is visualised in Figure 2.

There is a substantial class imbalance in the target variable, with 65.84% of ICOs in the dataset failing to meet their goals, compared to the 34.16% succeeding. This means that a naive classifier, which predicts all ICOs will fail, will achieve around 65.84% accuracy. Consequently, raw accuracy may be misleading in evaluation and model selection, and metrics less sensitive to class imbalance, such as F1 score and AUC, may be preferable.

The remaining indicator variables show inconsistencies, with multiple variants of positive and negative indicators, rather than the expected '1' and '0' classes.

Figure 3, visualises the distributions of the continuous variables in the dataset, excluding the largely missing `sold_tokens` and `pre_ico_price_usd` variables.

For the variables `price_usd`, `token_for_sale` and `distributed_in_ico`, a log scale was necessary to produce meaningful visualisations due to the significant positive skews of these predictors. Despite this log transformation, the histogram for `price_usd` still exhibits a significant positive skew, indicating the presence of extreme outliers. The median price per token in the dataset is \$0.15, with an interquartile range of \$0.67, whereas the maximum is \$828,486.02 which is unreasonably large in the context of ICOs, indicating that these large prices are error which should be removed. The `rating` variable exhibits an approximate normal distribution centred around 3. All values are within the expected 1-5 range, consistent with the defined rating scale. The `teamsize` histogram is positively skewed, with a median of 9, and the majority of

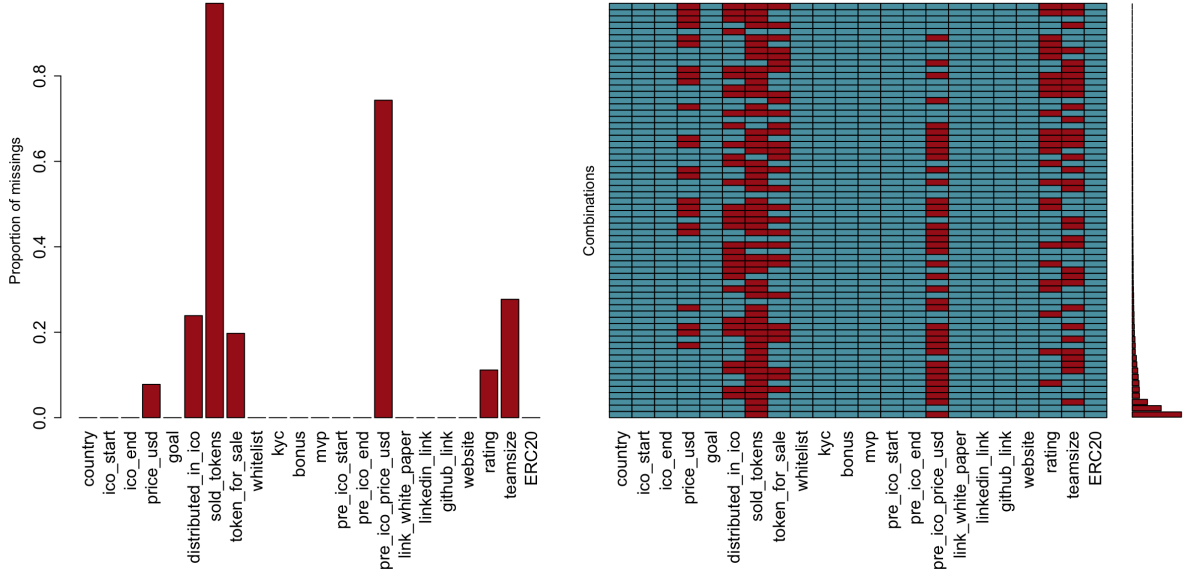


Figure 1: Bar and combinations plot showing the proportion of missing values by variable (left) and co-occurrence patterns of missingness across variable combinations (right) in the raw ICO dataset. Red indicates missing values; blue indicates observed values.

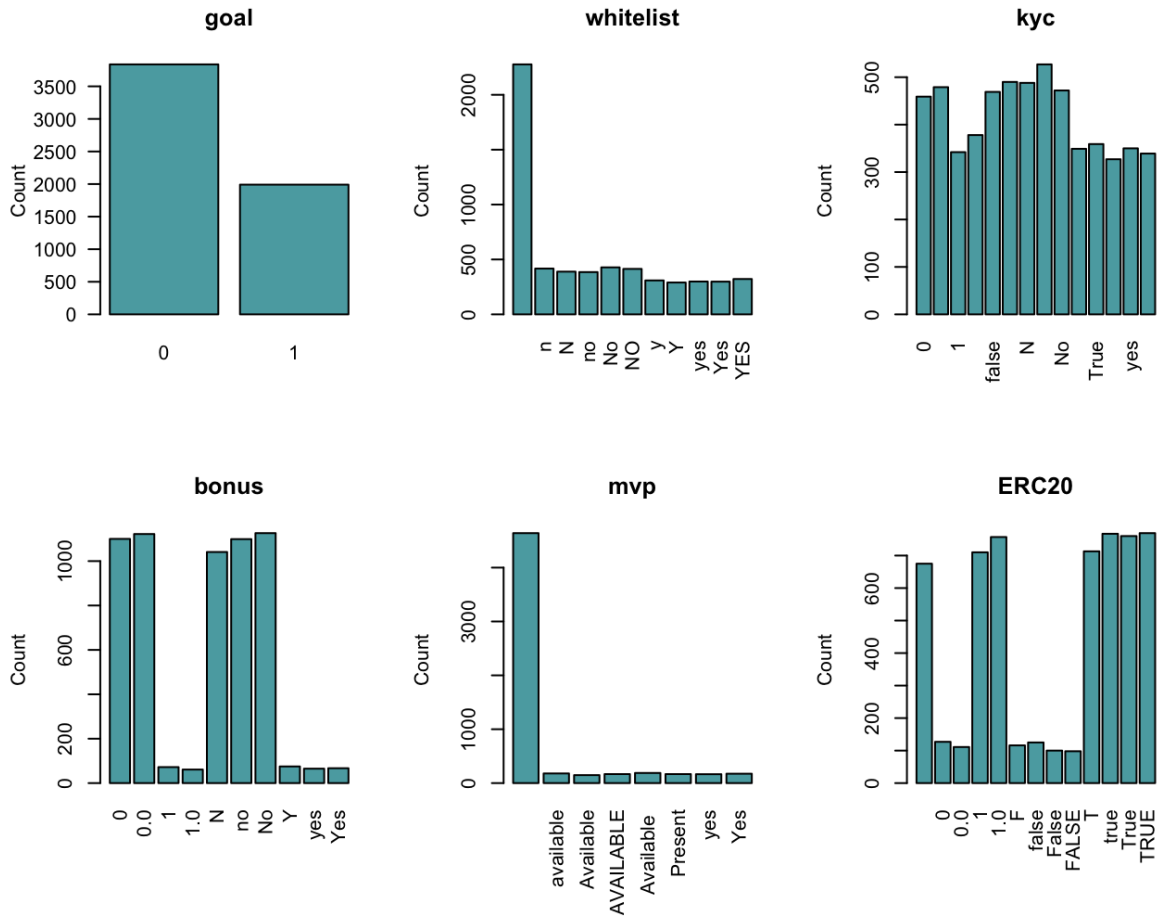


Figure 2: Bar plots showing the distribution of binary variables in the raw ICO dataset.

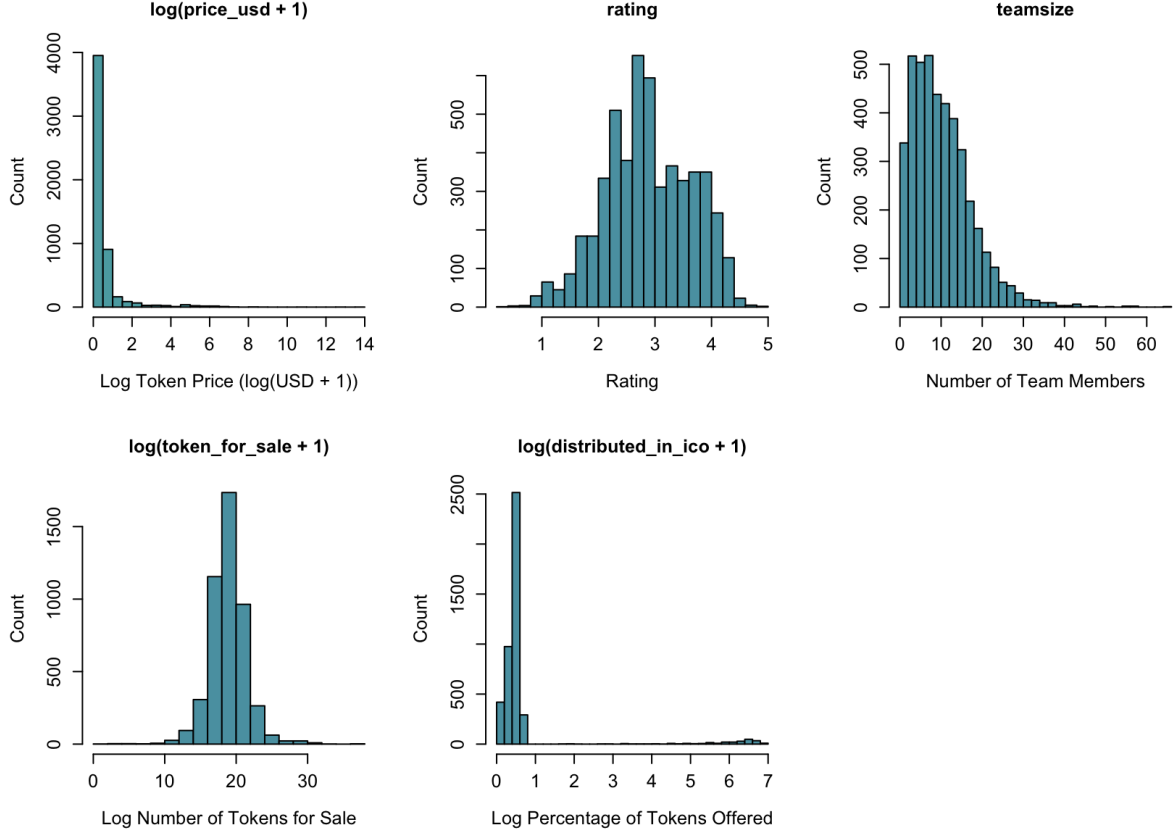


Figure 3: Frequency histograms of continuous predictor variables in the raw ICO dataset. Log transformations were applied to *price_usd*, *token_for_sale* and *distributed_in_ico*.

recorded ICOs consisting of fewer than 20 team members. The log-transformed *token_for_sale* histogram has a roughly normal distribution, suggesting this variable may have a log-normal distribution, thus, the raw data is positively skewed. Finally, the large spike near 0 in the log scaled *distributed_in_ico* histogram, and the small median of 0.56, suggest that many values may have been entered as proportions rather than percentages. The small spikes at 6-7 (around 400-1100 in the raw data), and maximum value of 995, indicate incorrect values as percentages can not exceed 100.

The country variable contained 143 unique string values, with varying capitalisation and trailing white space. Finally, the four date variables - *ico_start*, *ico_end*, *pre_ico_start* and *pre_ico_end* - were stored as character strings in day-month-year format rather than as date objects, requiring parsing prior to data analysis. Initial inspection also revealed implausible values in *ico_start*, pre-dating the ICO market, confirming data error in this variable.

4 Data Preparation

4.1 Handling Incorrect Values and Outliers

In order to prepare the dataset for modelling, the indicator variables - *whitelist*, *kyc*, *mvp*, *bonus* and *ERC20* - were cleaned in to binary variables, with positive indicator 1, and negative indicator 0.

The four date columns - *ico_start*, *ico_end*, *ico_pre_start*, *ico_pre_end* - were stored as character strings in day-month-year format. These were parsed to R date objects using the `lubridate` package for data manipulation. Rows with errors in these variables were identified as those with the end date of sale or pre-sale preceding the corresponding start date, i.e. those with negative sale or pre-sale durations. These 50 incorrect rows were list-wise deleted from the dataset. The maximum sale and pre-sale durations (3722 and 714 days) were significantly larger than their medians (40 and 30 days), suggesting extreme outliers. Manual inspection of campaigns with durations exceeding 365 days revealed systematic date entry errors. For example, row 10 records an end date of 2020-12-31 for a campaign beginning 2018-

05-01, implying a duration of 975 days. Cross-referencing with the ICO tracking platform ICOholder (icoholder.com/en/satt-21400) confirmed the actual end date for this SaTT token was 2019-03-31, indicating the year was incorrectly recorded. Similar errors were identified across other long-duration records, providing empirical justification for removing the 83 campaigns with durations or pre-durations exceeding 365 days as these likely represent data entry errors.

As discussed in the previous section, the *price_usd* variable contained implausibly large values, and a much larger mean (\$227.91) than median (\$0.15), and thus the 56 rows where the price was above the 99th percentile (\$236.48) were considered outliers, likely representing data entry errors, and were deleted.

Values less than 1 in the *distributed_in_ico* were assumed to be incorrectly recorded as proportions, rather than the desired percentages, and thus were multiplied by 100. Values exceeding 100 were considered errors and list-wise deletion was implemented on these 199 rows. After this initial cleaning, the dataset consisted of 5433 ICO campaigns.

Although the number of tokens in an ICO can greatly vary, the maximum value in the raw dataset is extremely large (2.260e+16), suggesting these outliers may be errors in the dataset. Thus, values in *token_for_sale* exceeding the 99th percentile (1e+12) were deleted. After this initial cleaning, the dataset consisted of 5389 ICO campaigns.

4.2 Feature Engineering

A binary dummy variable, *had_pre_ico*, was created to indicate when an ICO had a pre-sale offering period. This variable takes the value 0 if both the pre-sale start and end were missing for that row, and 1 otherwise.

A *duration* variable was created as the difference between the start and end sale dates (*ico_start - ico_end*), and a *pre_duration* variable was similarly derived using the pre-sale start and end columns, then setting values to 0 for rows where *had_pre_ico* indicated that this ICO did not have a pre-sale. These duration variables replaced the date columns, as numerical variables are better suited to predictive modelling.

As *token_for_sale* was observed to have an approximately log-normal distribution, this variable was replaced with the log transformed variable, *log_token_for_sale*, for modelling.

In order to use the URL variables in modelling, *link_white_paper*, *linkedin_link*, *github_link*, and *website* were each replaced with dummy variables - *has_whitepaper*, *has_linkedin*, *has_github* and *has_website* - with a value of 1 if URL was present in the dataset, and 0 otherwise.

The country variable contained messy character string values, difficult to model, and thus was replaced with a dummy variable, *is_US_Sing_UK* taking a positive value of 1 if the ICO was headquartered in the 3 most prevalent countries - US, UK or Singapore constitute 32.80% of ICOs - and taking the negative 0 value otherwise.

Finally, a variable taking the price of Bitcoin at the start date of the ICO sale was created as a proxy for the overall cryptocurrency market sentiment, as macro market conditions will likely influence the outcomes of ICOs. This was derived by merging the ICO dataset with the Bitcoin closing price, on the *ico_start* date, sourced from Yahoo Finance via the *quantmod* package.

4.3 Handling Missing Values

Following data cleaning and feature engineering, missingness in the remaining modelling variables was assessed. Figure 4 presents the distribution and co-occurrence of missing values across all variables.

The *sold_tokens* and *pre_ico_price_usd* variables were largely missing (96.81% and 74.69% respectively), and were thus deemed insufficient for modelling and these columns were dropped.

Missingness in the *btc_price* column was propagated from missing *ico_start* values, and thus strongly related to missingness in *duration*, which resulted from missing values in either *ico_start* or *ico_end*. As these variables are important in modelling, both highly positively skewed with extreme ranges, and the missingness in both variables overlaps and is not substantial (13.12% and 10.50% respectively), list-wise deletion was implemented, removing 707 rows from the dataset.

The variable *pre_duration* exhibited an extremely small proportion of missingness (0.08%) and thus list-wise deletion was also used to remove the 4 rows with missing values in this column. List-wise deletion was also selected for *price_usd*, which exhibited a relatively small 8.08% missingness after initial cleaning, removing a further 203 rows.

Missingness in the remaining variables (*rating*, *log_token_for_sale*, *teamsize* and *distributed_in_ico*) appeared to be independent rather than systematic, indicating that these variables are Missing Completely At Random (MCAR). *log_token_for_sale*, *teamsize* and *distributed_in_ico* exhibited missingness of 20.38%,

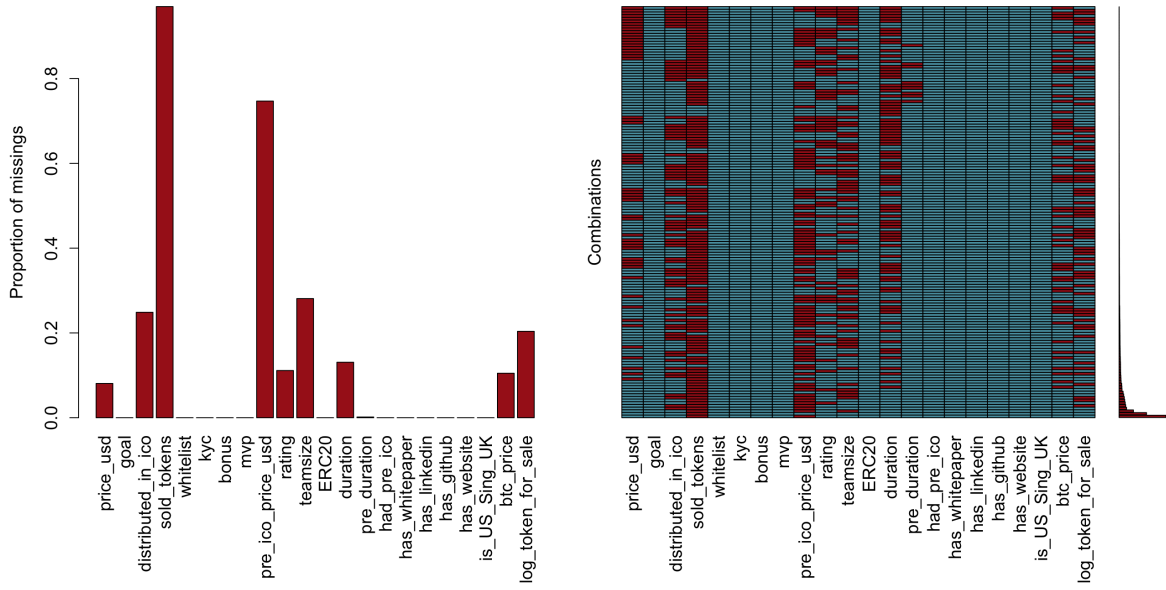


Figure 4: Bar and combinations plot showing the proportion and co-occurrence of missing values across modelling variables following initial data cleaning and feature engineering. Red indicates missing; blue indicates observed.

28.11% and 24.87% respectively, meaning list-wise deletion would result in severe data loss, thus imputation was selected. Although *rating* exhibited less severe missingness at 11.14%, its approximate normal distribution, observed in Figure 3, and narrow range (1-5) render this variable suitable for imputation. Median imputation was preferred over mean due to the generally skewed variable distributions. Figure 5 presents the effects of this missing data handling on these four variables.

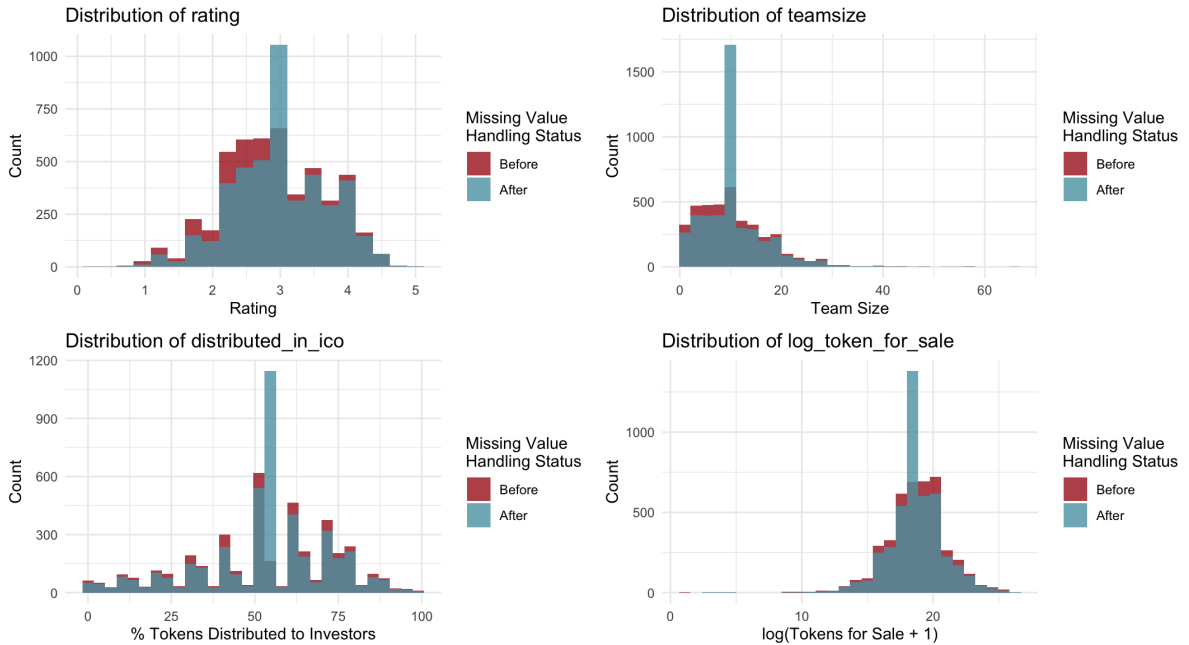


Figure 5: Frequency histograms of imputed continuous variables before (red) and after (blue) median imputation.

The median imputation caused a central spike in the distributions and reduced the standard deviations of these variables, as expected. However, due to the minimal skew in the four variables and the choice of

median over mean, after the initial cleaning, the effects on the variables' means and medians is minimal. Table 2 summarises the distributional properties of all continuously valued variables before and after missing value handling.

Variable	Method	Mean Before	Mean After	Median Before	Median After	SD Before	SD After
log_token_for_sale	Imputation	18.73	18.75	18.83	18.83	2.54	2.29
distributed_in_ico	Imputation	52.27	52.84	53.00	54.00	21.67	19.08
rating	Imputation	2.92	2.99	2.90	2.90	0.76	0.70
teambsize	Imputation	10.70	10.62	9.00	10.00	7.30	6.32
duration	Deletion	59.34	59.73	37.00	39.00	58.21	58.48
pre_duration	Deletion	17.61	19.13	0.00	0.00	34.43	35.26
price_usd	Deletion	2.50	2.51	0.15	0.15	14.90	15.01
btc_price	Deletion	7720.45	7744.10	7419.29	7419.29	3110.44	3124.64

Table 2: Summary statistics and handling method of continuous modelling variables before and after missing value handling.

Handling missing values has not significantly altered the summary statistics, and thus the underlying distributions, of the continuous variables, suggesting that this deletion and imputation has not introduced bias. As the binary variables did not exhibit missingness, the effects of missing data handling on their class distribution was negligible.

After handling incorrect values and outliers, engineering features, and handling missing values, the cleaned data set consists of 4475 rows and 19 variables.

4.4 Relationships Between Variables

Figure 6 visualises the relationship between the binary variables and the target variable *goal*.

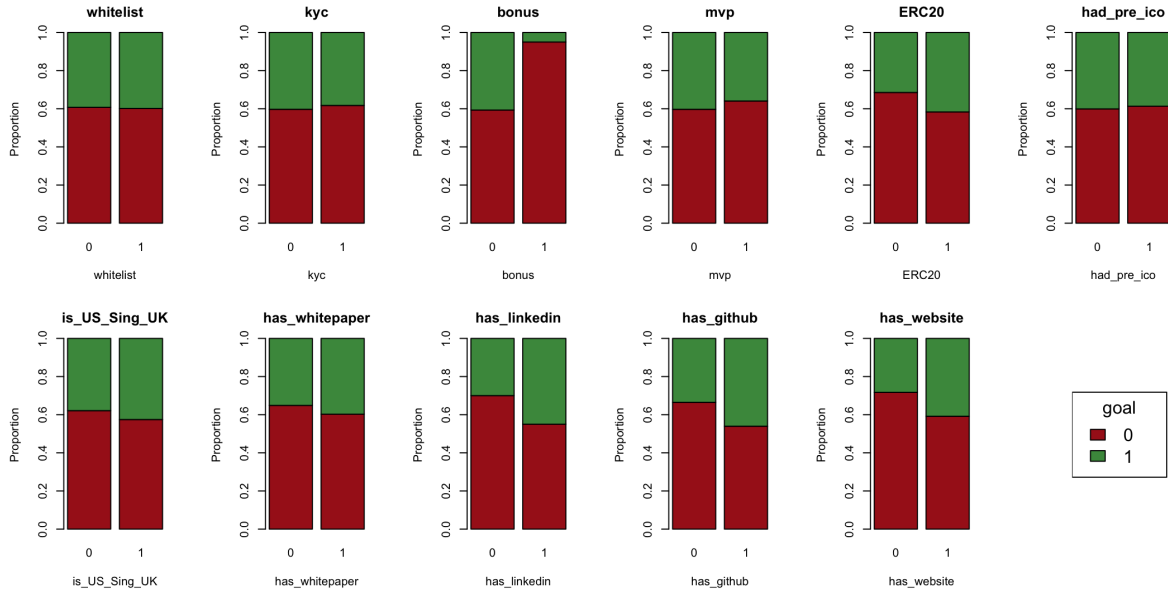


Figure 6: Stacked barplots showing the proportion of failed (red, 0) and successful (green, 1) ICO outcomes by class for each binary predictor variable in the cleaned dataset.

The variables *whitelist*, *kyc* and *had_pre_ico* show almost identical distributions of ICO outcome by class, suggesting these variables will not be important in predicting whether an ICO will achieve its goal. The URL indicator variables *has_whitepaper*, *has_linkedin*, *has_github* and *has_website* show a notable difference in ICO outcome by class, suggesting that transparency may be significant in whether an ICO project meets its target. *bonus* presents the most significant disparity - 95.00% of ICOs with information about bonuses failed, compared to 59.33% failing in cases where this information was not

present. However, this is likely due to the class imbalance in this variable: only 3.59% of projects in the dataset provided information on bonuses provided to token buyers.

Figure 7 presents how the distributions of the continuous variables vary based on the outcome of the ICO.

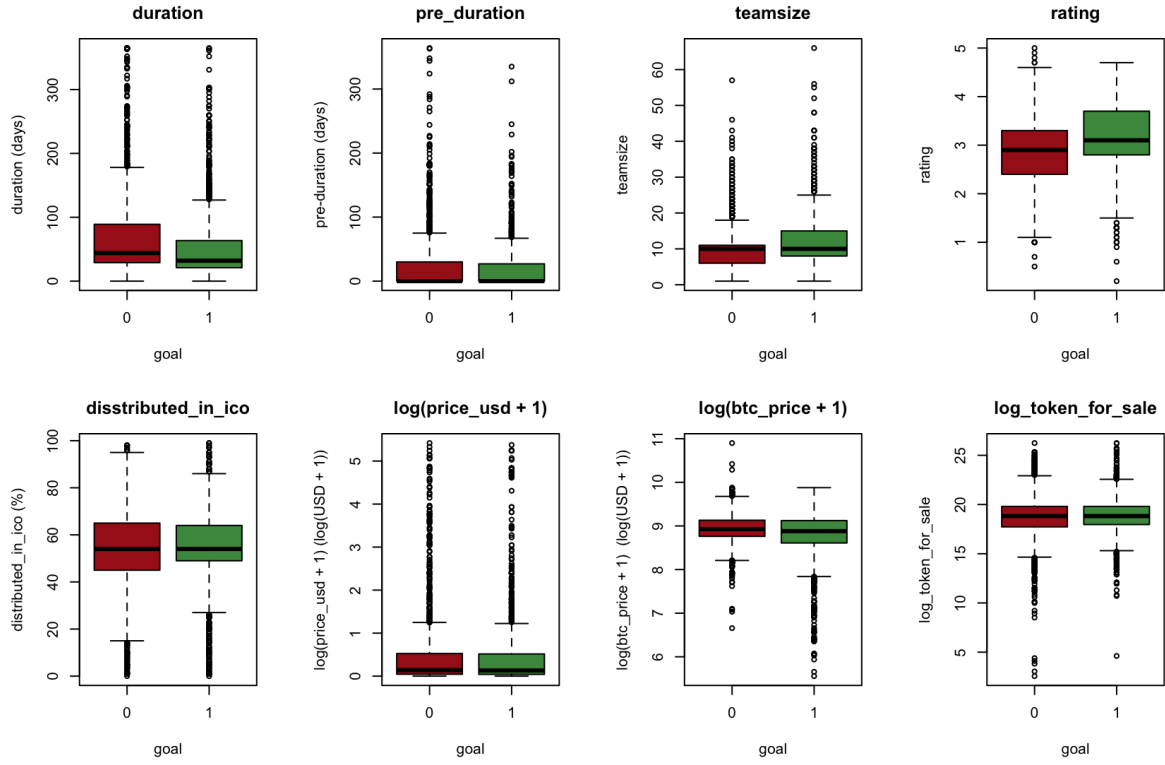


Figure 7: Boxplots comparing the distribution of continuous predictor variables between failed (red, 0) and successful (green, 1) ICO campaigns in the cleaned dataset.

The most striking differences are observed in rating and teamsize, which show a positive relationship with ICO success, and duration which exhibits a negative association with ICO outcome - successful ICOs generally show slightly shorter token offering periods, larger team sizes, and higher ratings than those which fail - suggesting these variables may be of greater importance in the proceeding predictive modelling.

To investigate the relationship between continuous predictors, correlation coefficients were calculated, visualised in Figure 8.

The correlation matrix revealed no strongly correlated continuous predictors, with *rating* and *teamsize* exhibiting the strongest relationship, with a coefficient of only 0.33. This suggests multicollinearity is not a concern for the proceeding modelling stage.

5 Modelling

Before modelling, the cleaned ICO dataset was first divided into a train and test set in an 80:20 ratio, ensuring reproducibility through a fixed random seed (123). The target variable, *goal*, was converted into a factor to enable classification modelling. After the split, the class distribution of the *goal* variable was checked in both subsets to confirm that the proportions of successful and failed ICOs remained consistent (approximately 61% 1s to 39% 0s), preventing sampling bias.

To prevent overfitting during hyperparameter tuning, 10-fold cross-validation was applied to the training set, further partitioning it into ten equal folds. For each hyperparameter configuration, the model was trained on 9 folds and evaluated on the remaining fold, with this process repeated 10 times. Mean AUC and across folds was used as the selection criterion, as this metric effectively assesses models with uneven target variable class distributions. Final model performance was evaluated on the held-out test set, which was not used in training and validation.

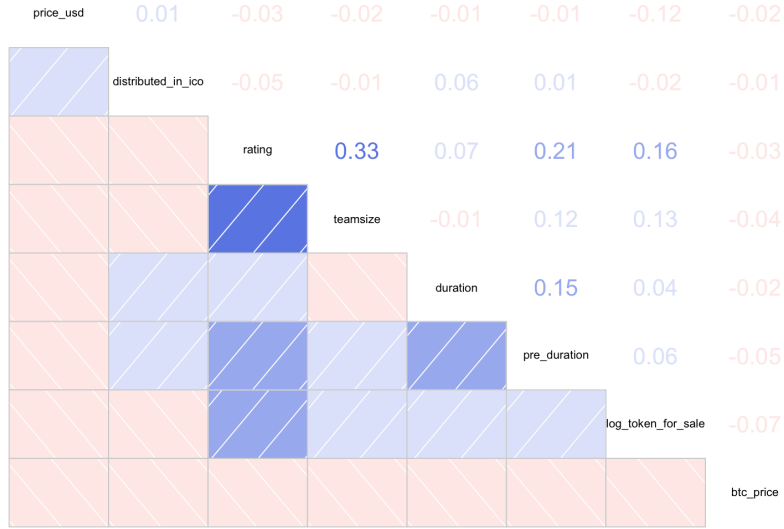


Figure 8: Corrgram visualising pairwise correlation coefficients, displayed numerically in the upper right panels, between continuous predictor variables in the cleaned ICO dataset. Blue shading indicates positive correlation; red shading indicates negative correlation.

Four supervised learning algorithms were selected for model development: K-Nearest Neighbours (KNN), Decision Tree, Random Forest, and Support Vector Machine (SVM).

5.1 K-Nearest Neighbour

The KNN algorithm was selected as it offers a simple and effective predictive classifier model which can be used as a baseline model. The algorithm's decision making process is highly interpretable, as its predictions are a result of the majority vote of a data points K spatial neighbours. In order to implement this distance based model, the train and test sets were z-score normalised using the mean and standard deviation of the training set to prevent data leakage. The K value of the final model was determined using 10-fold cross-validation, evaluating the mean AUC when K ranges from 1 to 121 in intervals of 10. These results are presented in Figure 9. The plots for both metrics display an elbow at around $K = 31$, but still show some variation until plateauing when K is around 61. Thus, although AUC peaked at a K value of 111, the final KNN model was created using $K = 61$ to produce high scoring metrics while avoiding unnecessary model complexity and high computational testing cost.

5.2 Decision Tree

The Decision Tree model was selected as it offers a highly interpretable classification model, which is valuable in this context, as it enables understanding as to why a campaign is predicted to succeed.

The algorithm was implemented using the `C5.0` Decision Tree classifier function in R, with adaptive boosting trials. The number of boosting trials was tuned via 10-fold cross-validation, evaluating AUC across trials = 1 to 20. Figure 10 presents AUC as a function of adaptive trials. AUC rose steeply from around 0.695 at trials 1-3 to 0.728 at 7 trials, peaking at 0.734 when trials = 16. Thus, the final model was trained with 16 adaptive boosting trials.

5.3 Random Forest

The Random Forest algorithm was also selected for this analysis. As it is an ensemble method, averaging a large number of decision trees built on bootstrapped subsamples of the data, it offers stability, robustness to noise in the data, and interpretable feature importance estimates.

This algorithm was implemented using the `randomForest` function in R, from the package of the same name. The number of trees was selected via evaluating AUC through 10-fold cross-validation, the

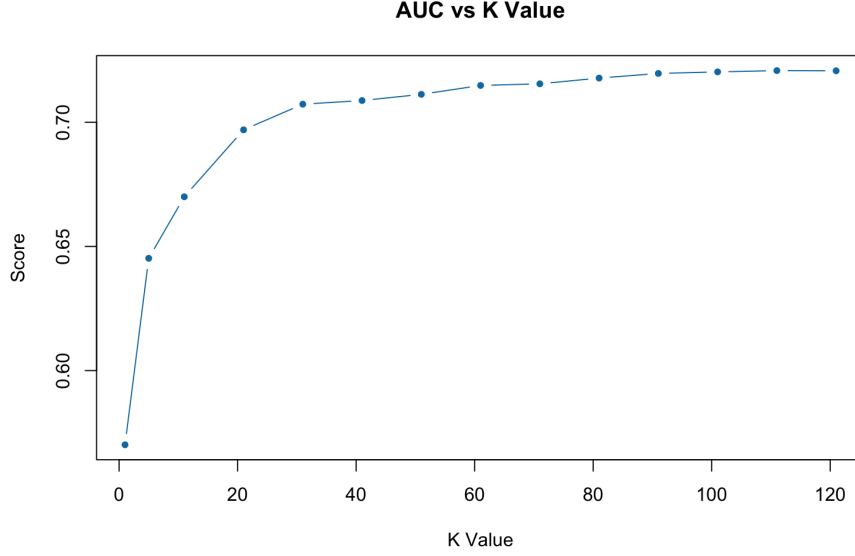


Figure 9: KNN mean AUC score across 10 cross-validation folds as a function of the number of neighbours K , ranging from 1 to 121 in intervals of 10.

results of which are visualised in Figure 11. The AUC generally increases with number of trees, peaking at 0.745 when $n_{trees} = 400$. Thus, the final Random Forest model was trained with 400 trees on the whole train dataset.

Figure 12 presents the variable importance plot from the final Random Forest model. *rating*, *duration* and *teamsize* ranked among the most important predictors for both mean decrease in accuracy and mean decrease in Gini impurity, along with *btc_price*, supporting the inclusion of market sentiment as a predictor.

5.4 Support Vector Machine

Support Vector Machine was selected due to its effectiveness in high dimensional space. Through the use of different kernels, SVMs offer flexibility and the capability to model non-linear data.

The algorithm was implemented using the `ksvm` function in R, and the cost (penalty for misclassification) and kernel type hyperparameters were selected via 10-fold cross validation. Cost values of $C = 0.01, 0.1, 1, 10$ were evaluated in combination with linear (`vanilladot`), RBF (`rbfdot`) and polynomial (`polydot`) kernels, the results of which are presented in Table 3.

	Cost	Kernel	Mean AUC	SD AUC
1	0.010	vanilladot	0.709	0.025
2	0.010	rbfdot	0.716	0.025
3	0.010	polydot	0.709	0.025
4	0.100	vanilladot	0.710	0.024
5	0.100	rbfdot	0.721	0.024
6	0.100	polydot	0.710	0.024
7	1.000	vanilladot	0.710	0.024
8	1.000	rbfdot	0.724	0.027
9	1.000	polydot	0.710	0.024
10	10.000	vanilladot	0.710	0.024
11	10.000	rbfdot	0.695	0.028
12	10.000	polydot	0.710	0.024

Table 3: Mean AUC and standard deviation across 10 cross-validation folds for each combination of cost parameter C and kernel type evaluated in SVM hyperparameter tuning.

Mean AUC was maximised with a cost value of 1 and the `rbfdot` kernel, suggesting that non-linear

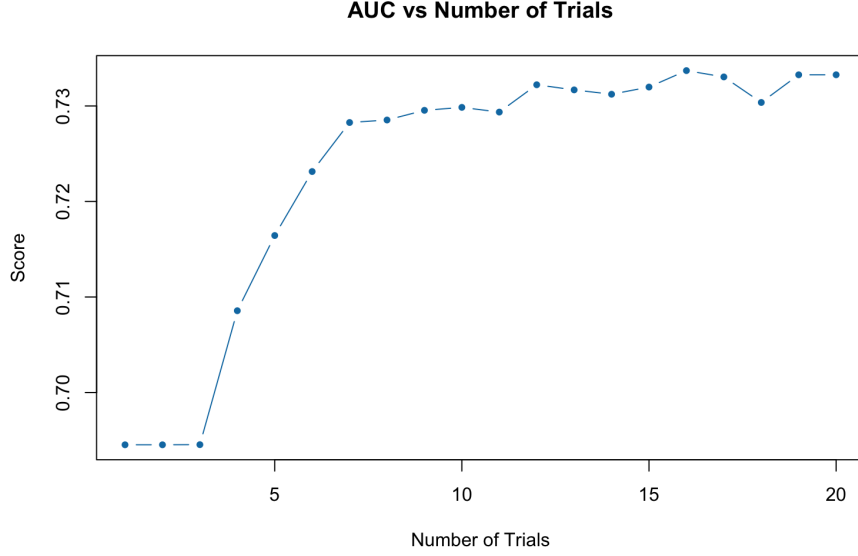


Figure 10: Decision Tree mean AUC score across 10 cross-validation folds as a function of the number of adaptive boosting trials, ranging from 1 to 20.

decision boundaries better characterise the separation between successful and failed ICOs.

6 Evaluation

Table 4 presents the performance of each final model on the held-out test set across four metrics.

	Accuracy	Precision	Recall	F1
DecisionTree	0.676	0.638	0.439	0.520
RandomForest	0.685	0.660	0.439	0.527
SVM	0.726	0.694	0.536	0.605
KNN	0.675	0.645	0.416	0.506

Table 4: Performance of each final model on the held-out test set across accuracy, precision, recall and F1 score.

As discussed previously, the class imbalance in the dataset (39% positive class) means that accuracy alone is insufficient as a selection metric. This imbalance has the consequence of all models being biased toward predicting failure, which can be seen in the consistently lower recall scores (proportion of true positives correctly identified) compared to precision (proportion of predicted positives that are true positives). The F1 score, the harmonic mean of precision and recall, penalises models that achieve high accuracy by predominantly predicting the majority class (a model predicting all failures achieves $F1 = 0$ despite 60% accuracy) making it a more meaningful indicator of predictive power for this imbalanced dataset. SVM achieves the highest F1 score, substantially outperforming the other three models. However, F1 is dependent on strict classification threshold of 0.5, whereas the ROC curve and AUC offer a more comprehensive, threshold independent, model evaluation. The ROC curves and AUC scores of each model are visualised in Figure 13. All four models sit comfortably above the diagonal, confirming that they provide genuine predictive ability beyond random chance. The curves are broadly similar in shape, scoring within a narrow range of AUC scores (0.719-0.732), indicating that the predictive ceiling is driven primarily by the volatile nature of ICO outcomes and the limited power of the dataset, rather than the type of model family. However, Random Forest achieves the highest AUC, and achieves higher sensitivity at lower specificity values, with its curve sitting above the rest in the right hand side of the graph. This indicates that at lower thresholds, the Random Forest model is the best predictor for whether an ICO will meet its target.

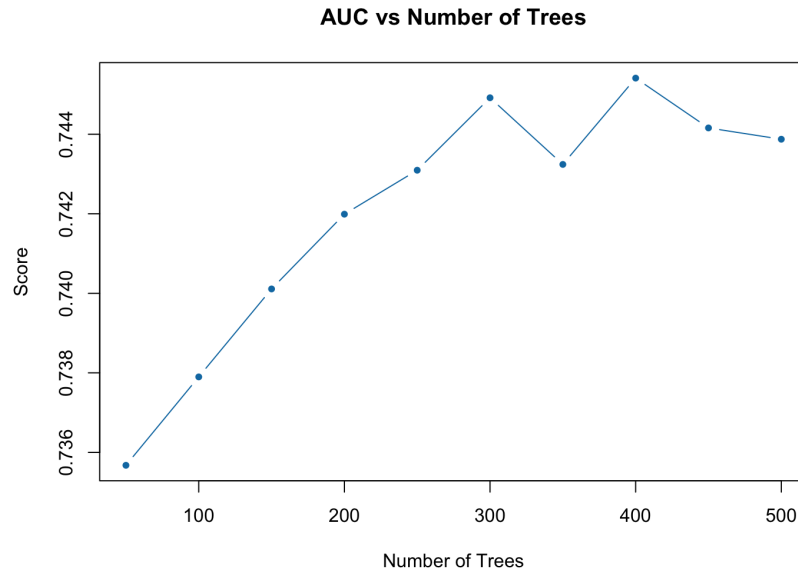


Figure 11: Random Forest mean AUC score across 10 cross-validation folds as a function of the number of trees, ranging from 50 to 500 in periods of 50.

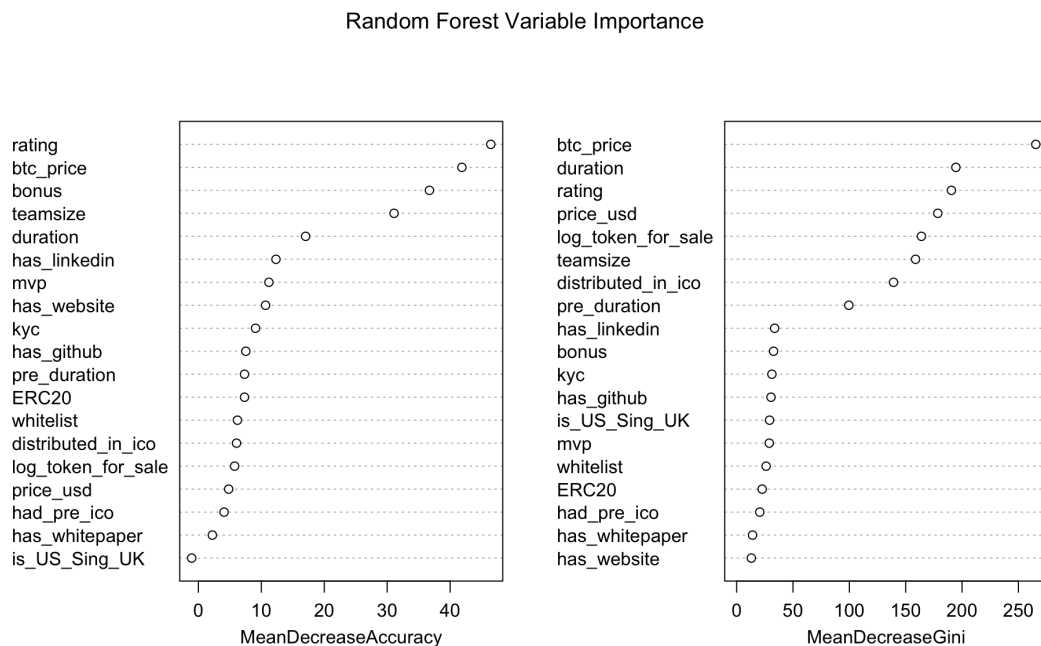


Figure 12: Variable importance plot from the final Random Forest model, showing mean decrease in accuracy (left) and mean decrease in Gini impurity (right) for each predictor variable.

7 Conclusion

This report developed machine learning models to predict ICO fundraising success using a dataset of 5,828 campaigns sourced from IcoBench. After extensive data cleaning and feature engineering, four models were trained and evaluated: KNN, Decision Tree, Random Forest and SVM.

Whilst Random Forest achieved the highest AUC (0.733), SVM produced the strongest performance across the classification metrics at the default 0.5 threshold, achieving the highest accuracy, precision, recall and F1 score, indicating that SVM is the preferable model when a simple success vs failure prediction label is required at the fixed 0.5 threshold, for example if investors request a binary decision whether to

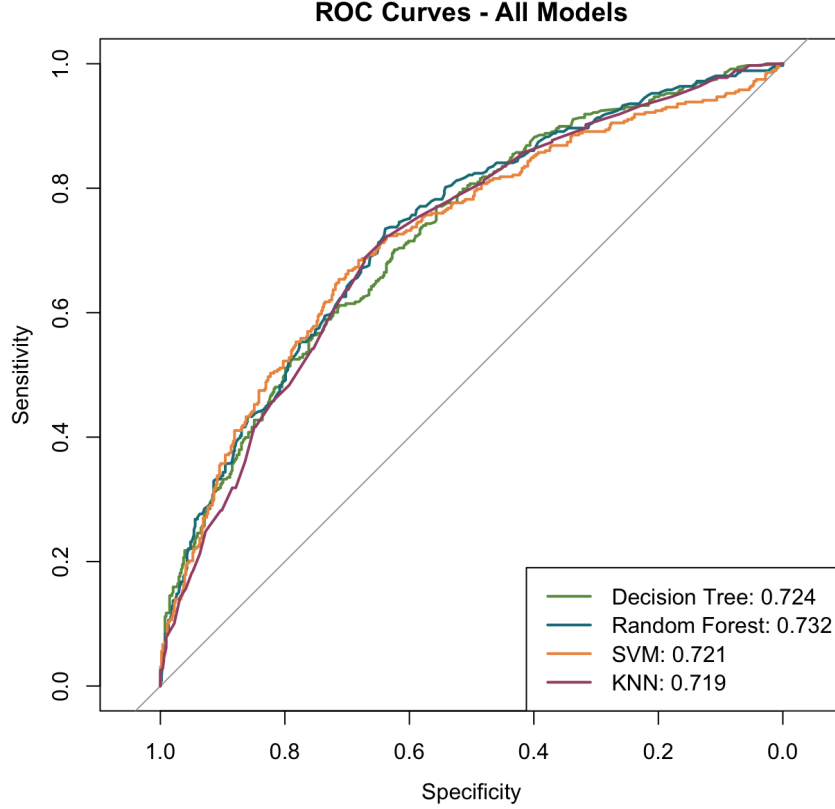


Figure 13: ROC curves for all four final models evaluated on the held-out test set. The diagonal grey line represents a random classifier (AUC = 0.5). AUC scores are displayed in the legend.

purchase tokens for a particular ICO. Whereas, the Random Forest model is superior when the goal is to rank the probability of ICOs succeeding, for example, if investors want a ranked list of ICOs to compare their likelihood of reaching their target.

The variable importance plot from the final Random Forest model indicates that expert rating, bitcoin price at the launch date, and team size are among the strongest predictors for ICO success. This is aligned with Huang et al. (2021), who find that team size is statistically significantly positively associated with the amount an ICO raises. The importance of the *btc_price* variable reflects Momtaz's (2020) assertion that market sentiment is significantly positively related to ICO first day returns.

The presence of a whitepaper, LinkedIn, GitHub and website URL showed a positive association with ICO success in exploratory analysis of the cleaned dataset, suggesting that project transparency reduced investor uncertainty and increases the likelihood of an ICO project meeting its targets.

Thus, when planning ICO campaigns, companies should consider the macro market sentiment, ensure they have sufficient team members, achieve a high expert rating, and focus on communicating with transparency to potential investors. Investors should consider these same factors when selecting which ICOs to select, prioritising highly rated projects, with a large number of team members, available URLs and launched during positive market conditions.

Limitations of this analysis include the bias towards predicting ICO failure in all four models as a result of the class imbalance in the dataset. Future work could apply resampling techniques to address this imbalance. Furthermore, the extensive missingness in the dataset meant variables had to be dropped and extensive list-wise deletion and imputation has to be implemented, potentially introducing bias and limiting the dataset, and a more complete dataset with fewer missing values would likely improve performance. The inclusion of more variables, external to the ICObench dataset, may also meaningfully improve the predictive power of the model. For example, Huang et al. (2021) demonstrated that perceived management confidence is significantly positively associated with the amount of money raised for an ICO project. Finally, extending the dataset beyond 2021 would improve the generalisability of predictive models to current and future ICO projects.

References

- [1] Huang, W., Vismara, S. and Wei, X., 2021. Confidence and capital raising. *Journal of Corporate Finance*, p.101900.
- [2] Momtaz PP (2020) Initial Coin Offerings. *PLoS ONE* 15(5): e0233018.